API REFERENCE MANUAL

AI RAG Chatbot Method Signatures & Symbol Maps

Technical Code Definitions

Detailed structural index of core python modules, classes, and parameter lists.

DOCUMENTED API PACKAGES:

- auth: Authentication logic & SQLite users setup
- rag: Document loading, chunking, embedding, vector store & retrieval
- llm: Groq API client integration & conversation state trimming
- utils: DB sessions management & history logging helpers

Generated: July 02, 2026 | Version 1.0.0 Stable
Document Type: Technical API Reference

1. Authentication, Parsing & Processing Modules

Authentication Module (auth.auth)

File Path: auth/auth.py

-> hash_password(plain: str) -> str
Hashes a plaintext credential with bcrypt using a random salt.

-> verify_password(plain: str, hashed: str) -> bool
Checks password string inputs against stored database hashes.

-> create_user(username: str, password: str, email: str = None) -> dict
Inserts records to SQLite user table, validating uniqueness.

-> authenticate_user(username: str, password: str) -> Optional[dict]
Checks username existence and password hashes, returning metadata.

Document Parser Pipeline (rag.loader)

File Path: rag/loader.py

-> load_file(filepath: str) -> str
Main dispatcher. Determines type and invokes internal parser (PDF, DOCX, CSV, TXT, MD).

-> _load_pdf(path: str) -> str
Extracts text across PDF pages using pdfplumber, joining them with double-newlines.

Text Chunker (rag.chunker)

File Path: rag/chunker.py

-> chunk_text(text: str, chunk_size: int = 500, overlap: int = 50) -> List[str]
Splits documents into overlapping chunks based on word boundaries.

2. Embeddings, Vector Search & Database Connectors

Embedding Engine (rag.embedder)

File Path: rag/embedder.py

-> get_model() -> SentenceTransformer
Loads the embeddings model as an in-memory singleton. Defaults to 'all-MiniLM-L6-v2'.

-> embed_texts(texts: List[str]) -> List[List[float]]
Vectorizes batches of text strings into 384-dimensional dense arrays.

ChromaDB Connector (rag.vector_store)

File Path: rag/vector_store.py

-> add_chunks(chunks: List[str], embeddings: List[List[float]], source: str, doc_id: str)
Upserts batches of text blocks and float arrays to ChromaDB.

-> query_chunks(query_embedding: List[float], top_k: int = 4) -> List[dict]
Executes similarity searches based on Cosine distances, returning context and scores.

LLM Controller (llm.groq_client)

File Path: llm/groq_client.py

-> chat(messages: List[dict], context: str = '', stream: bool = True) -> Generator[str]
Configures prompts and pipes inputs to LLaMA 3.3 on Groq. Handles chunk streaming.

-> trim_history(messages: List[dict]) -> List[dict]
Limits dialog payloads to the last 20 records while preserving user-assistant message pairs.

SQL Database Adaptor (utils.helpers)

File Path: utils/helpers.py

-> init_db() -> None
Initializes relational tables documents and chat_history. Automatically performs username migrations.

-> get_history(session_id: str, username: str = None) -> List[dict]
Retrieves the conversation logs for a session, filtering by username if provided.